

Assignment 5: Heaps and Priority Queues, Graphs, & Hashing

Madeline LeBreton

T00785871

COMP 2231

Submitted 04/09/2025

- I. Make the heap sort algorithm more efficient by writing a method that will build a heap in place using the array to be sorted.**

Testing my implementation:

```
madelinelebreton@MacBookAir assignment 5 part 1 % java TestHeapSort

Test case 1: [9023, 1029, 984, 783, 645, 234, 222, 102, 92, 37, 22, 12, 9, 6, 3, 0] (in order)
Results after heapSort: [9023, 1029, 984, 783, 645, 234, 222, 102, 92, 37, 22, 12, 9, 6, 3, 0]

Test case 2: [34, 56, 67, 2, 3, 7, 123, 45, 85, 434, 2345, 45, 765, 5, 0] (random)
Results after heapSort: [2345, 765, 434, 123, 85, 67, 56, 45, 45, 34, 7, 5, 3, 2, 0]

Test case 3: [1, 3, 5, 7, 9, 12] (reverse order)
Results after heapSort: [12, 9, 7, 5, 3, 1]

Test case 4: [33, 56, 7, 91, 7, 3, 4, 7, 0, 1, 3, 0, 7, 0, 4] (with duplicated)
Results after heapSort: [91, 56, 33, 7, 7, 7, 7, 4, 4, 3, 3, 1, 0, 0, 0]

Test case 5: [35] (single element)
Results after heapSort: [35]

Test case 6: [] (empty list)
Results after heapSort: []
```

- II. Complete the implementation of Graph.java using an adjacency matrix**

Testing my implementation:

```
madelinelebreton@Madelines-MacBook-Air-2 assignment 5 part 2 % java TestGraph

Testing addVertex and size methods

Created a new empty graph. Size: 0
Adding A, B, and C as vertices...

New size (3 expected): 3

Vertices: [A, B, C]

Testing addVertex() with empty string...

Size (4 expected): 4

Vertices: [A, B, C, ]
Removing empty vertex.
Vertices: [A, B, C]
```

```
Testing addEdge and removeEdge methods
Adding edge between A and B
Adding edge between B and C
Is the graph connected? (expect true): true
Removing edge between A and B
Is the graph connected? (expect false): false

Testing removeVertex method
Size before (3 expected): 3
Removing vertex B
Size after (2 expected): 2
Index of B (-1 expected): -1
Vertices (expect A and C): [A, C]

Testing removeVertex by index
Removing vertex at [1]
Size after (1 expected): 1
Index of C (-1 expected): -1
Vertices (expect A only): [A]

Testing getIndex method
Index of A (0 expected): 0
Index of C (-1 expected): -1
Index of Z (-1 expected): -1

Testing indexIsValid method
Adding back vertices B and C for testing
Size after (3 expected): 3
Vertices (expect A, B, C): [A, B, C]
Is index 0 valid (expect True): true
Is index 1 valid (expect True): true
Is index 2 valid (expect True): true
Is index 3 valid (expect False): false
Is index 10 valid (expect False): false

Testing getVertices method
getVertices() call (expect A, B, C): [A, B, C]

Testing isConnected method on single vertex
Removing vertices B and C
getVertices() call: [A]
isConnected() call (expect true, trivially connected): true
```

```

Testing iteratorBFS method and DFS
Adding vertices B, C, D, E for testing
Vertices: [A, B, C, D, E]
Adding edges (A, B), (A,C), and (B,D)

Creating BFS iterator from A
bfs.next(): (expect A)  A
bfs.next(): (expect B)  B
bfs.next(): (expect C)  C
bfs.next(): (expect D)  D
bfs.hasNext(): (expect false)  false

Creating DFS iterator from A
dfs.next(): (expect A)  A
dfs.next(): (expect B)  B
dfs.next(): (expect D)  D
dfs.next(): (expect C)  C
dfs.hasNext(): (expect false)  false

Testing removeEdge method on invalid indices
Size before: 5
Vertices: [A, B, C, D, E]
Calling removeEdge(0,5)
Size after: 5
Vertices: [A, B, C, D, E]
Calling removeEdge(-1,1)
Size after: 5
Vertices: [A, B, C, D, E]

End of testing

```

III. Implement a dynamically resizable hash table to store book names and 10-digit ISBN numbers

Testing my implementation:

```

madelinelebreton@Madelines-MacBook-Air-2 assignment 5 part 3 % java TestHashTable

Testing my implementation of a Hash Table

Initial insertions (collisions expected)
Putting 1234567890, 9876543890, 1112222890 as Book A, B, and C

Putting 5556667771 and 2223334771 as Book D and E

```

```
Current table:
Hash Table Contents:
0: 9876543890: Book B
1: 1112222890: Book C
2: 5556667771: Book D
3: 2223334771: Book E
4: null
5: null
6: null
7: null
8: null
9: null
10: 1234567890: Book A

Testing get() method
Get 1234567890: Book A
Get 9876543890: Book B
Get 0000000000 (not in table): null

Testing remove() method
Remove 9876543890: true
Remove 0000000000 (not in table): false
```

```
Current table:
Hash Table Contents:
0: 1112222890: Book C
1: 5556667771: Book D
2: 2223334771: Book E
3: null
4: null
5: null
6: null
7: null
8: null
9: null
10: 1234567890: Book A

Current size:11
Adding more books to trigger resize
Size after:22
```

```
Current table:
Hash Table Contents:
0: null
1: 5556667771: Book D
2: 2223334771: Book E
3: 1111111111: Book F
4: 2222222222: Book G
5: 3333333333: Book H
6: 4444444444: Book I
7: 5555555555: Book J
8: null
9: null
10: 1112222890: Book C
11: 1234567890: Book A
12: null
13: null
14: null
15: null
16: null
17: null
18: null
19: null
20: null
21: null
Testing get() method after resize
Get 5555555555: Book J
Get 1112222890: Book C
Testing remove after resize
Remove 1234567890: true
```

```
Current table:
Hash Table Contents:
0: null
1: 5556667771: Book D
2: 2223334771: Book E
3: 1111111111: Book F
4: 2222222222: Book G
5: 3333333333: Book H
6: 4444444444: Book I
7: 5555555555: Book J
8: null
9: null
10: 1112222890: Book C
11: null
12: null
13: null
14: null
15: null
16: null
17: null
18: null
19: null
20: null
21: null
```

The same set of values can result in different hashes, depending on order in which they are added

Insertion order: A, B, C

Hash Table Contents:

0: 9876543890: B
1: 1112222890: C
2: null
3: null
4: null
5: null
6: null
7: null
8: null
9: null
10: 1234567890: A

Insertion order: C, A, B

Hash Table Contents:

0: 1234567890: A
1: 9876543890: B
2: null
3: null
4: null
5: null
6: null
7: null
8: null
9: null
10: 1112222890: C